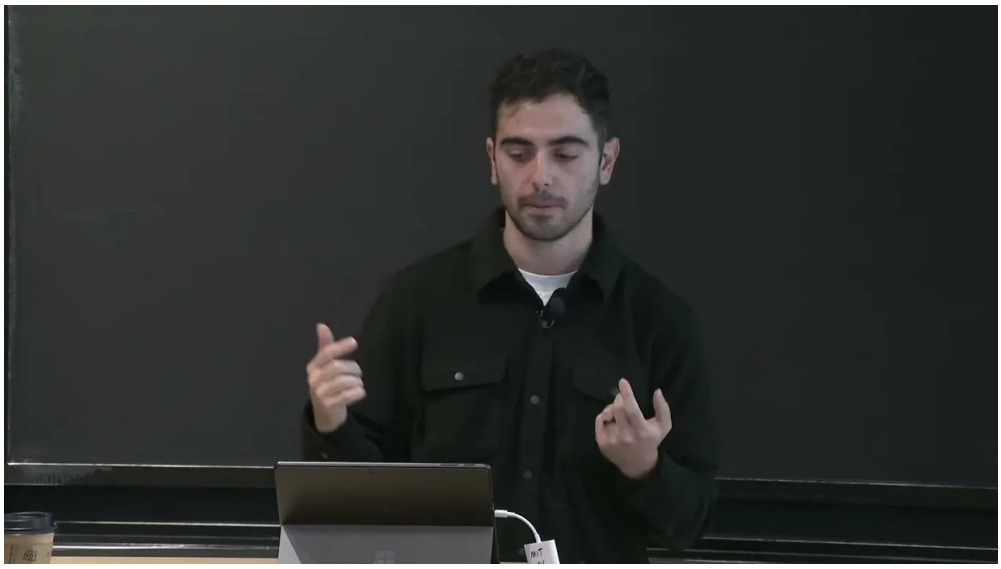
课程笔记

MIT Neuroblox 教材笔记 06：神经元、神经团块与源

MIT Neuroblox 课程组 & Codex

2026 年 4 月 15 日



视频作者/频道：MIT Video Productions
发布日期：2025-04-10；课程标题标注 2025-01-15
视频时长：12 分 54 秒
视频链接：`https://www.youtube.com/watch?v=Ptqv16fhOtg`

目录

1 导论：学习目标与对象总览

本讲把 Neuroblox 的三类基础对象放在同一张地图上：*神经元*、*神经团块*（*neural mass*）和 *源*（*source*）。视频开场先明确：在前面几讲已经讲过 Julia 和 Neuroblox 的编程语法之后，这一讲开始回到神经科学本体，先看最基本的神经元，再看群体平均后的神经团块，最后看如何用外部源驱动模型。

这堂课真正关心的不是“画什么图”，而是“变量如何通信”

Neuroblox 中三类对象的核心差别，不是名字，而是它们如何把自己的状态送进下游方程：

- 神经元通常通过连续状态或事件触发来影响别的 Blox；
- 神经团块把群体行为压缩成更少的状态变量，强调平均活动、同步/去同步和参数区间；
- 源本身没有输入，只负责向下游注入连续驱动、脉冲列或刺激协议。

这也是本讲所有例子的共同骨架。

对象	通信方式	代表例子	适合回答的问题
Neuron	连续状态 / 事件触发	QIF, HH, Izhikevich	单细胞兴奋性、发放、
Neural mass	群体平均 / 振荡器 / 解析导出	Wilson-Cowan, Jensen, next generation	平均活动、同步状态、
Source	仅输出、无输入	ConstantInput, PoissonSpikeTrain, DBS	外部驱动、实验协议、

表 1: 本讲三类 Blox 的接口差异。

课程页在首页就把学习目标写得很直接：先可视化神经元与神经团块的仿真结果，再把外部源作为 Blox 引入，最后用这些源去驱动单神经元与群体活动。这个目标虽然朴素，但它实际上定义了后面所有 API 的使用边界：先建模，再连接，再看输出。

1.1 本章小结

本节只做了框架搭建：后面的所有细节，都可以看成是在回答同一个问题——“不同层级的神经模型，究竟通过什么接口把状态送到下游?”

2 神经元：单元动力学、发放率与随机性

神经元部分先从生物结构讲起：树突接收输入，胞体整合信号，轴突负责把动作电位传给别的神经元。课程视频特意强调了一个数量级事实：人脑里神经元数量极大，而且多数神经元并不是孤立工作的，树突平均会接收到很多其他神经元的输入。这个背景解释了为什么单神经元模型既要足够简洁，又要保留“输入–整合–输出”这条基本链路。

动作电位在课堂里被描述成一个从化学信号转成电信号的过程：突触前释放递质，递质进入突触间隙，被突触后受体接收，受体打开离子通道，离子流入引发突触后电位。也就是说，单神经元模型里最关键的不是“会不会放电”这个结论，而是受体、通道、离子流和膜电位之间怎样协同。

2.1 Hodgkin-Huxley 的意义：不仅是生物细节，也是等效电路

课程视频把 Hodgkin-Huxley 模型放在很核心的位置：它不仅刻画了受体动力学和膜电位变化，还提供了一个等效电路视角。这个视角很重要，因为它把“生物过程”翻译成了“受体、电容、电源”组成的电路，使得单神经元可以既像生物系统，又像一个可计算的动力学系统。

```
using Neuroblox
using OrdinaryDiffEq
using CairoMakie
using Random
Random.seed!(1)

@named qif = QIFNeuron(; I_in=4)
sys = system(qif; discrete_events = [60] => [qif.I_in ~ 10])
tspan = (0, 100) # ms
prob = ODEProblem(sys, [], tspan)
sol = solve(prob, Tsit5())

fig = rasterplot(qif, sol; threshold=-40)
fig = frplot(qif, sol; threshold=-40, win_size=20)
V = voltage_timeseries(qif, sol)
fig = lines(V)

using StochasticDiffEq
@named hh = HHNeuronExci_STN_Adam_Blox(; sigma = 2)
sys = system(hh)
prob = SDEProblem(sys, [], (0, 1000))
sol = solve(prob, RKMil())
fig = powerspectrumplot(hh, sol; sampling_rate=0.01)
```

Listing 1: QIF 与 HH 的标准仿真接口示意。

上面的代码块把本讲最重要的三种可观测量放在一起了：`rasterplot` 看离散放电，`frplot` 看滑动窗意义下的发放率，`powerspectrumplot` 则把随机扰动下的频谱结构显示出来。课程页还特地指出，随机 HH 神经元的模型中含有 brownian noise 项，它会改变频谱上的能量分布，所以只看时域波形是不够的。

U V Z¥6` bi2`ThQi U#V Z¥6`BM;@` ih2 THQi

mR, ZA*6Üí¥ ñÅ˜VjÄj˜b[ñ:ÜñÌ¥ B ¨Ÿªüo^Ž?bpÐo?b
qMÄp^ ª,]• b

mk,Û >>*Üí¥ÿqöb[ñ:¨ñ4œ € .2i,º^o îpñö×ÑÔö² b

kXk AMi2;`ÐiQ2bQM iQ6ÀwL,^ em

jÔ¤"Ü*Üísî Ë BMi2;` ÐQ``2bQMb[iQ:¨ 6A +mÜp=2€us7Ÿb
BMi2;`¥iQ$`^oÿÆ È@ Ü(?bq Úp `2bQM÷i^Q$`ñ{•´¥71 t
{ÆS¶=+¬,?b tV ñuWªŒÉÆüAbÈ ÿb
ƒB ¥1o,^sË 7^[ñjÔ<Ø¥ #B7m`+4iÄBQEMñ*Ü ˜ YV¿M
•" V[á a«p?ba #m`abti?b-WMÐbÐ Åª 6AwL„s´m,^¸c 7

是神经元动力学的相图。

2.3 Izhikevich 模型的教学价值

视频用 Izhikevich 作为例子，原因很朴素：二维系统足够简单，所以可以真的把很多 regime 画出来。课程页也延续这个思路，用 QIF / Izhikevich 类模型演示 raster、firing rate 和电压轨迹。相比 HH 这样更细的生物模型，Izhikevich 的优势是参数和变量更少，便于把“模型能做什么”先看清楚；代价则是生物解释力更弱。

这一讲对单神经元模型的态度

更简单的模型不一定更“假”。如果你的研究问题是发放模式、阈值效应、分岔或噪声如何改变群体行为，那么更小的模型常常更有解释力，因为你能直接看见它的 regime 变化。

2.4 本章小结

神经元部分真正传递的是一个方法论：先把单细胞动力学抽象成可计算的状态，再用合适的可视化去读它的 regime。发放率曲线、raster、电压轨迹和频谱分别回答不同的问题，不能互相替代。

3 神经团块：平均场、振荡器与下一代模型

神经团块模型比单神经元更上一层抽象楼梯。它们要描述的不是某个细胞逐毫秒的膜电位，而是群体平均活动、同步性和振荡结构。视频里给出的第一个例子是 Wilson-Cowan，它用兴奋/抑制两个变量就能描述一个神经群体的平均 E-I 平衡。

```
using Neuroblox
using OrdinaryDiffEq
using CairoMakie
using Random
Random.seed!(1)

@named nm = WilsonCowan()
sys = system(nm)
tspan = (0, 100) # ms
prob = ODEProblem(sys, [], tspan)
sol = solve(prob, Tsit5())

fig, ax = plot(sol)
axislegend(ax)
fig

E = state_timeseries(nm, sol, "E")
fig = lines(E)
fig
```

Listing 2: Wilson-Cowan Blox 的标准仿真与状态提取。

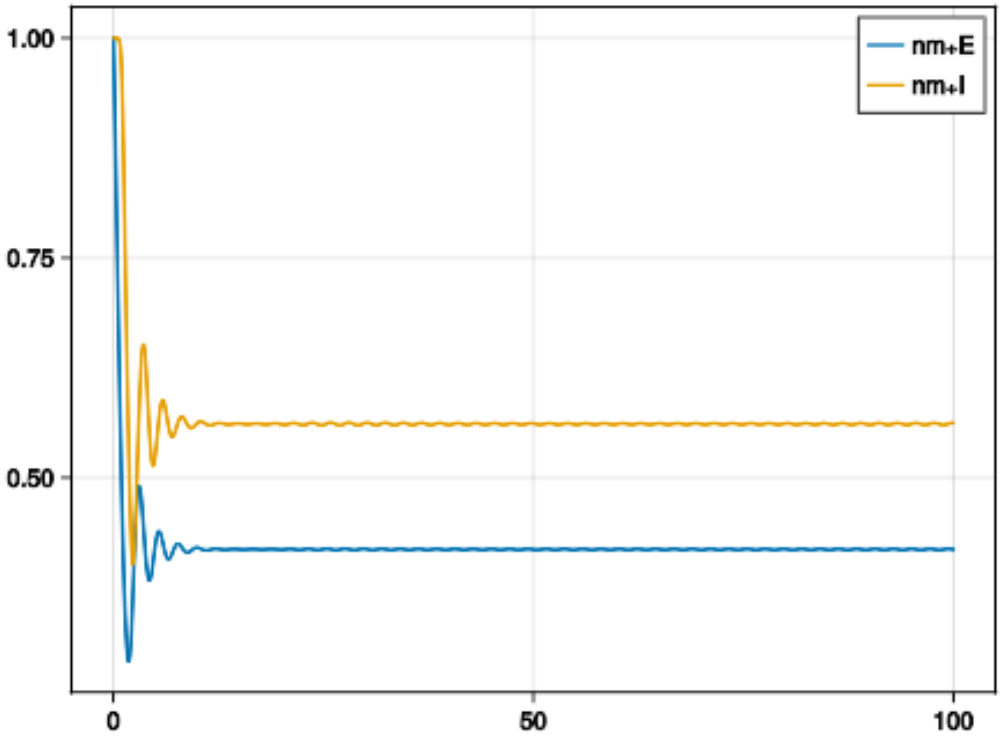


图 4: ConstantInput 连接后 Wilson-Cowan 的输出。输入把原先的 E-I 平衡整体推移了。

这张图的教学意义很直接：source 不只是 “多加一个常数”，而是可以把整个系统的平衡点拖到新的位置。视频也明确指出，后续课程会用更复杂的 circuit 来继续研究 E-I balance 的变化。

4.2 事件触发 spike 源

下一类 source 是事件触发式的 spike source。课程页先给出 PoissonSpikeTrain：先定一个时间区间 `tspan`，再给固定 spike rate；或者把 rate 设成一个带 `distribution` 和 `dt` 的结构，让 rate 自己也随时间随机变化。

```
tspan = (0, 200) # ms
spike_rate = 0.01 # spikes / ms
@named spike_train_rate = PoissonSpikeTrain(spike_rate, tspan)

using Distributions
spike_rate = (distibution = Normal(1, 0.1), dt = 10)
@named spike_train_dist = PoissonSpikeTrain(spike_rate, tspan)
```

Listing 4: 固定速率与随机速率的 PoissonSpikeTrain。

课程页接着给出一个自定义事件源的完整示例：`BernoulliSpikes`。实现思路很清楚：先定义 source 结构体，再重载 `generate_spike_times` 来生成 spike 时间，最后重载 `connection_spike_affects` 来指定每次 spike 到来时如何修改下游神经元的方程。

```
struct BernoulliSpikes <: SpikeSource
    name
    namespace
    tspan
```

```
    probability_spike
    dt
    function BernoulliSpikes(probability_spike, tspan, dt; name, namespace=nothing)
        new(name, namespace, tspan, probability_spike, dt)
    end
end

import Neuroblox: generate_spike_times, connection_spike_affects

function generate_spike_times(source::BernoulliSpikes)
    t_range = source.tspan[1]:source.dt:source.tspan[2]
    t_spikes = Float64[]
    for t in t_range
        if rand(Bernoulli(source.probability_spike))
            push!(t_spikes, t)
        end
    end
    return t_spikes
end

function connection_spike_affects(source::BernoulliSpikes, ifn::IFNeuron, w)
    eqs = [ifn.I_in ~ ifn.I_in + w]
    return eqs
end
```

Listing 5: BernoulliSpikes 与事件回调的自定义示例。

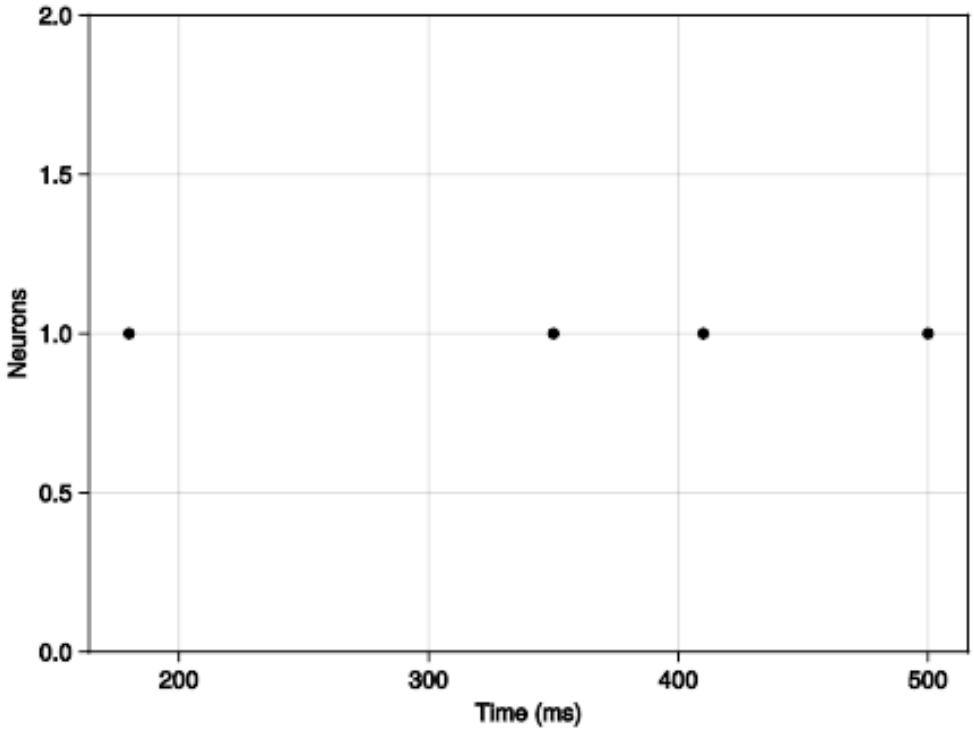


图 5: Bernoulli / Poisson 风格 spike source 驱动 IF 神经元的输出。